

Executed Notebook Run

MAE / MFE Trade Click Notebook

XAUUSD | May 25-29, 2026 | Carryover trade excluded

Notebook source: C:\Users\HP\Downloads\mae_mfe_click_trade_chart_2026-05-25_29.ipynb

Note: interactive click behavior is preserved in the notebook; this PDF contains static snapshots of the run.

Notebook Run Summary

Tick coverage: 2026-05-25 01:00:00.591000 to 2026-05-29 22:59:59.927000

Raw closed positions: 363; excluded carryover positions: 1; included positions: 362

Calculated MAE/MFE rows: 362

Cell Output: Excluded Carryover Trade

original_trade_id	entry_time	exit_time	type	entry_price	exit_price	sl	tp	profit
0	2026-05-22 22:49:00	2026-05-25 01:00:00	sell	4507.31	4509.81	4509.81	4503.81	-2.5

Cell Output: First Included Trades

trade_id	entry_time	exit_time	type	entry_price	exit_price	sl	tp	profit
0	2026-05-25 02:48:00	2026-05-25 02:56:46	sell	4576.68	4573.18	4579.18	4573.18	3.5
1	2026-05-25 03:11:00	2026-05-25 03:12:25	buy	4575.56	4572.92	4572.92	4578.92	-2.64
2	2026-05-25 03:18:00	2026-05-25 03:23:18	sell	4570.42	4572.92	4572.92	4566.92	-2.5
3	2026-05-25 03:25:00	2026-05-25 03:25:40	buy	4574.72	4572.22	4572.22	4578.22	-2.5
4	2026-05-25 03:28:00	2026-05-25 03:30:00	sell	4570.36	4572.86	4572.86	4566.86	-2.5

Summary Tables

Cell Output: Summary Metrics

included_trades	net_profit	win_rate_pct	avg_mae_points	median_mae_points	avg_mfe_points	median_mfe_points
362	23.44	39.5	186.59	203.5	205.77	187.0

Cell Output: MAE/MFE Quantiles

quantile	mae_points	mfe_points
0.1	42.2	0.0
0.25	99.75	43.0
0.5	203.5	187.0
0.75	257.0	358.75
0.9	277.9	382.0
0.95	294.9	412.5

Cell Output: Winner vs Loser Summary

	mae_points_count	mae_points_mean	mae_points_median	mae_points_min	mae_points_max	mfe_points_count	mfe_points_mean	mfe_points_median	mfe_points_min	mfe_points_max	profit_count	profit_mean	profit_median	profit_min	profit_max
loser	219	244.87	254.0	72.0	1276.0	219	95.14	64.0	0.0	345.0	219	-2.15	-2.5	-2.9	-0.26
winner	143	97.33	85.0	7.0	239.0	143	375.19	365.0	304.0	522.0	143	3.46	3.5	0.81	4.14

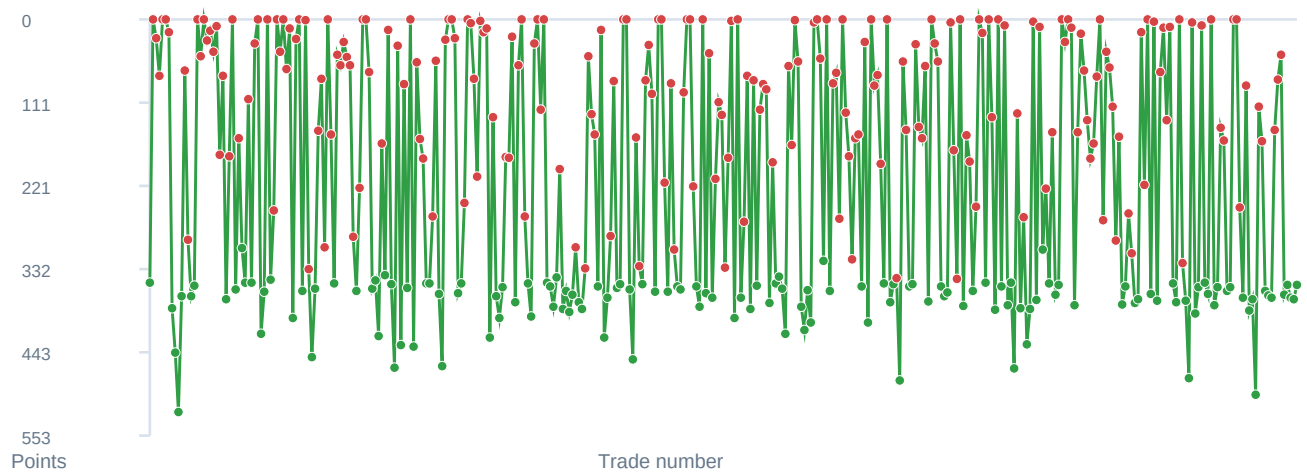
Static Plot Output

The live notebook has separate clickable MAE and MFE plots. This PDF captures static snapshots from the same run.

MAE by Trade



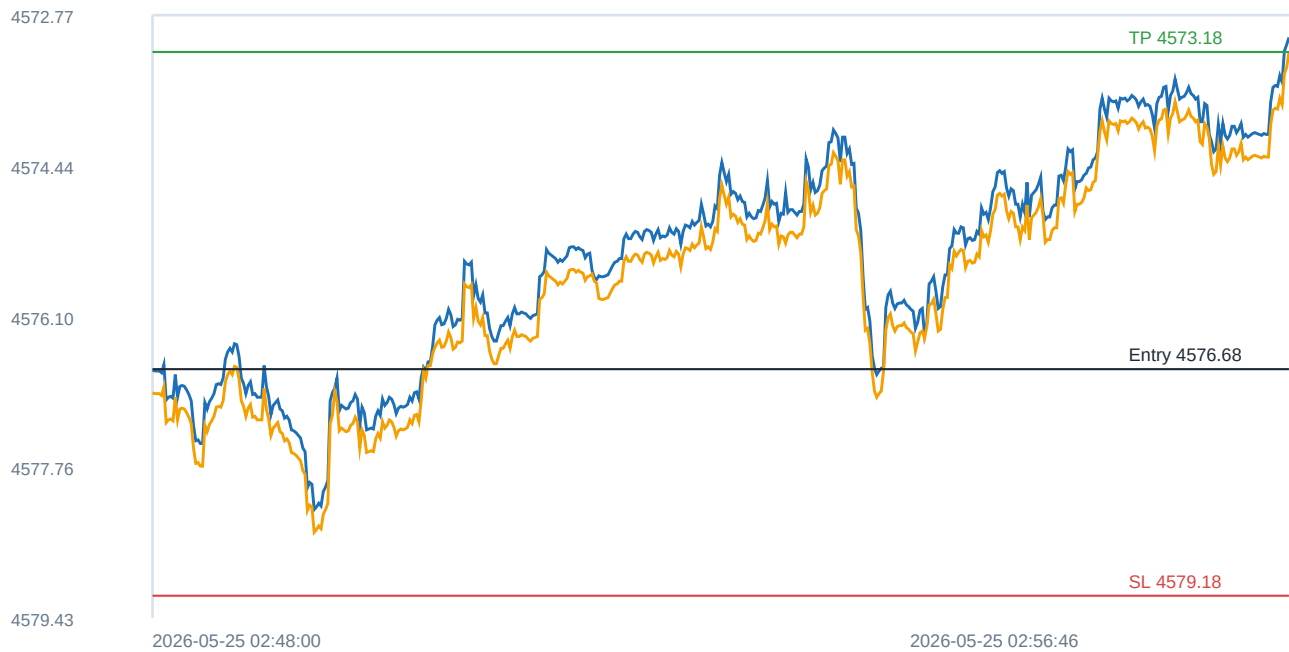
MFE by Trade



Selected Trade Chart Snapshot

Default selected trade from the notebook widget: trade 0, position 8768165665.

Selected trade chart: Trade 0 (SELL)



trade_id	type	entry_time	exit_time	profit	mae_points	mfe_points
0	sell	2026-05-25 02:48:00	2026-05-25 02:56:46	3.5	181.0	350.0

Notebook Code Reference

Cell 1

```

from pathlib import Path
import json
import math
import html
import warnings

import numpy as
np
import pandas as pd
try:
from IPython.display import HTML, display
except ImportError:

class HTML(str):
    pass

def display(value):
    if isinstance(value, HTML):

print("<HTML output omitted outside Jupyter>")
    else:

print(value)

warnings.filterwarnings("ignore", category=UserWarning, module="openpyxl")

NOTEBOOK_DIR = Path.cwd()
TICK_FILE = NOTEBOOK_DIR / "XAUUSD_week.csv"
REPORT_FILE =
NOTEBOOK_DIR / "ReportHistory-week.xlsx"
if not TICK_FILE.exists():
    TICK_FILE =
    Path(r"C:\Users\HP\Downloads\XAUUSD_week.csv")
if not REPORT_FILE.exists():
    REPORT_FILE =
    Path(r"C:\Users\HP\Downloads\ReportHistory-week.xlsx")

SYMBOL = "XAUUSD"
POINT_SIZE = 0.01

MAX_CHART_POINTS_PER_TRADE = 500

assert TICK_FILE.exists(), f"Missing tick file: {TICK_FILE}"

assert REPORT_FILE.exists(), f"Missing report file: {REPORT_FILE}"

```

Cell 2

```

def find_section(raw: pd.DataFrame, label: str) -> int:
    matches = raw.apply(lambda row:
        row.astype(str).str.strip().eq(label).any(), axis=1)
    idx = raw.index[matches].tolist()

    if not idx:
        raise ValueError(f"Could not find section {label!r}")
    return
    int(idx[0])

def load_mt5_positions(path: Path, symbol: str) -> pd.DataFrame:
    raw =
    pd.read_excel(path, sheet_name="Sheet1", header=None)
    positions_row = find_section(raw,
        "Positions")
    orders_row = find_section(raw, "Orders")

    rows = raw.iloc[positions_row +
        2 : orders_row].copy()
    rows = rows[rows[0].notna()].copy()
    rows = rows.iloc[:, :13]

    rows.columns = [
        "entry_time",
        "position",
        "symbol",

        "type",
        "volume",
        "entry_price",

```

```

"sl",
"tp",

"exit_time",
"exit_price",
"commission",
"swap",
"profit",

]

rows["symbol"] = rows["symbol"].astype(str).str.strip()
rows["type"] =
rows["type"].astype(str).str.lower().str.strip()
rows =
rows[rows["symbol"].eq(symbol)].copy()

rows["entry_time"] =
pd.to_datetime(rows["entry_time"], format="%Y.%m.%d %H:%M:%S", errors="coerce")

rows["exit_time"] = pd.to_datetime(rows["exit_time"], format="%Y.%m.%d %H:%M:%S",
errors="coerce")
for col in ["volume", "entry_price", "sl", "tp", "exit_price",
"commission", "swap", "profit"]:
rows[col] = pd.to_numeric(rows[col], errors="coerce")

rows = rows.dropna(subset=["entry_time", "exit_time", "entry_price", "exit_price",
"profit"])
rows = rows.sort_values(["entry_time", "position"]).reset_index(drop=True)

rows["original_trade_id"] = rows.index
return rows

def load_ticks(path: Path) ->
pd.DataFrame:
if path.stat().st_size < 50_000_000:
raise RuntimeError(

f"Tick file looks incomplete: {path} is only {path.stat().st_size:,} bytes. "
"Use
the full XAUUSD_week.csv export, expected around 86 MB."
)

ticks =
pd.read_csv(path, sep="\t")
ticks.columns = [str(c).strip().lower().replace("<",
 "").replace(">", "") for c in ticks.columns]
required = {"date", "time", "bid", "ask"}

missing = required.difference(ticks.columns)
if missing:
raise ValueError(f"Tick
file missing columns: {sorted(missing)}")

ticks["datetime"] = pd.to_datetime(

ticks["date"].astype(str) + " " + ticks["time"].astype(str),
format="%Y.%m.%d
%H:%M:%S.%f",
errors="coerce",
)
ticks["bid"] = pd.to_numeric(ticks["bid"],
errors="coerce")
ticks["ask"] = pd.to_numeric(ticks["ask"], errors="coerce")
ticks =
ticks.dropna(subset=["datetime", "bid", "ask"]).sort_values("datetime").reset_index(drop=True)

ticks = ticks[(ticks["bid"] > 1000) & (ticks["ask"] > 1000) & (ticks["ask"] >=
ticks["bid"])].copy()
if ticks.empty:
raise RuntimeError("No valid XAUUSD bid/ask
ticks were found after cleaning.")
ticks["mid"] = (ticks["bid"] + ticks["ask"]) / 2.0

return ticks[["datetime", "bid", "ask", "mid"]]

```

Cell 3

```

positions_raw = load_mt5_positions(REPORT_FILE, SYMBOL)
ticks = load_ticks(TICK_FILE)

tick_start = ticks["datetime"].min()
tick_end = ticks["datetime"].max()

excluded =

```

```

positions_raw[positions_raw["entry_time"] < tick_start].copy()
positions = positions_raw[

(positions_raw["entry_time"] >= tick_start)
& (positions_raw["exit_time"] <= tick_end)

].copy().reset_index(drop=True)
positions["trade_id"] = positions.index

print(f"Tick coverage:
{tick_start} to {tick_end}")
print(f"Raw closed positions: {len(positions_raw):,}")

print(f"Excluded carryover positions opened before tick coverage: {len(excluded):,}")

display(excluded[["original_trade_id", "entry_time", "exit_time", "type", "entry_price",
"exit_price", "sl", "tp", "profit"]])
print(f"Included positions for MAE/MFE:
{len(positions):,}")
display(positions.head())

```

Cell 4

```

def downsample_indices(length: int, max_points: int) -> np.ndarray:
    if length <=
max_points:
    return np.arange(length)
    return np.unique(np.linspace(0, length - 1,
max_points).round().astype(int))

def calculate_trade_excursions(trades: pd.DataFrame, ticks:
pd.DataFrame) -> pd.DataFrame:
    tick_times = ticks["datetime"].to_numpy()
    bid =
ticks["bid"].to_numpy(dtype=float)
    ask = ticks["ask"].to_numpy(dtype=float)
    mid =
ticks["mid"].to_numpy(dtype=float)
    records = []

    for row in
trades.itertuples(index=False):
        start = np.searchsorted(tick_times,
np.datetime64(row.entry_time), side="left")
        # MT5 report timestamps are second-
precision while ticks are millisecond-precision.
        # Include ticks inside the reported
exit second so stop/TP touches are not missed.
        exit_boundary = row.exit_time +
pd.Timedelta(seconds=1)
        end = np.searchsorted(tick_times, np.datetime64(exit_boundary),
side="left")
        if end <= start:
            continue

        bid_w = bid[start:end]

        ask_w = ask[start:end]
        mid_w = mid[start:end]
        time_w =
tick_times[start:end]

        if row.type == "buy":
            closeable = bid_w

        favorable = bid_w - row.entry_price
        adverse = row.entry_price - bid_w
        elif
row.type == "sell":
            closeable = ask_w
            favorable = row.entry_price -
ask_w
            adverse = ask_w - row.entry_price
        else:
            continue

        mae_points = max(0.0, float(np.nanmax(adverse)) / POINT_SIZE)
        mfe_points =
max(0.0, float(np.nanmax(favorable)) / POINT_SIZE)
        path_points = np.where(row.type ==
"buy", (bid_w - row.entry_price) / POINT_SIZE, (row.entry_price - ask_w) / POINT_SIZE)

    chart_idx = downsample_indices(len(time_w), MAX_CHART_POINTS_PER_TRADE)

```



```

records.append(
{
"trade_id": int(row.trade_id),

"original_trade_id": int(row.original_trade_id),
"position": str(row.position),

"symbol": row.symbol,
"type": row.type,

"volume": float(row.volume),
"entry_time": row.entry_time,

"exit_time": row.exit_time,
"duration_min": round((row.exit_time -
row.entry_time).total_seconds() / 60.0, 3),
"entry_price":
float(row.entry_price),
"exit_price": float(row.exit_price),

"sl": float(row.sl),
"tp": float(row.tp),
"profit":
float(row.profit),
"commission": float(row.commission),
"swap":
float(row.swap),
"mae_points": round(mae_points, 2),

"mfe_points": round(mfe_points, 2),
"max_adverse_price":
round(float(closeable[int(np.nanargmax(adverse))]), 5),
"max_favorable_price":
round(float(closeable[int(np.nanargmax(favorable))]), 5),
"min_path_points":
round(float(np.nanmin(path_points)), 2),
"max_path_points":
round(float(np.nanmax(path_points)), 2),
"chart_time":
pd.to_datetime(time_w[chart_idx]).strftime("%Y-%m-%d %H:%M:%S.%f").str[: -3].tolist(),

"chart_bid": np.round(bid_w[chart_idx], 3).tolist(),

"chart_ask": np.round(ask_w[chart_idx], 3).tolist(),
"chart_mid":
np.round(mid_w[chart_idx], 3).tolist(),
}
)

out =
pd.DataFrame(records)
return out.sort_values("entry_time").reset_index(drop=True)

trades
= calculate_trade_excursions(positions, ticks)
print(f"Calculated MAE/MFE for {len(trades):,}
trades")
display(trades[["trade_id", "entry_time", "exit_time", "type", "profit", "mae_points",
"mfe_points"]].head())

```

Cell 5

```

summary = {
"included_trades": len(trades),
"net_profit":
round(float(trades["profit"].sum()), 2),
"win_rate_pct": round(float((trades["profit"] >
0).mean() * 100), 2),
"avg_mae_points": round(float(trades["mae_points"].mean()), 2),

"median_mae_points": round(float(trades["mae_points"].median()), 2),
"avg_mfe_points":
round(float(trades["mfe_points"].mean()), 2),
"median_mfe_points":
round(float(trades["mfe_points"].median()), 2),
}

display(pd.DataFrame([summary]))

quantiles
= [0.10, 0.25, 0.50, 0.75, 0.90, 0.95]
mae_mfe_quantiles = trades[["mae_points",
"mfe_points"]].quantile(quantiles).rename_axis("quantile")
display(mae_mfe_quantiles)

display(

```

```
trades.groupby(trades["profit"].gt(0).map({True: "winner", False:
"loser"}))["mae_points", "mfe_points", "profit"]
.agg(["count", "mean", "median", "min",
"max"])
)
```

Cell 6

```
def build_click_widget(...):
# Builds separate clickable MAE and MFE plots plus selected
trade chart.
# Full interactive code remains in the .ipynb notebook.
```

Cell 7

```
processed_path = Path("mae_mfe_trades_2026-05-25_29.csv")
trades.drop(columns=["chart_time",
"chart_bid", "chart_ask", "chart_mid"]).to_csv(processed_path, index=False)
print(f"Saved
processed MAE/MFE table beside the notebook runtime as: {processed_path.resolve()}")
```